НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені Ігоря СІКОРСЬКОГО»
Навчально-науковий фізико-технічний інститут
Кафедра математичних методів захисту інформації

Звіт до лабораторної №3

за темою:
Дослідження криптографічних протоколів та механізмів захисту платіжних систем

Оформлення звіту:
Юрчук Олексій, ФІ-52мн

3 листопада 2026 р.
м. Київ

ЗМІСТ

1 Мета **1**

2 Механізми безпеки сучасних електронних платежів **1**

3 Карткові онлайн-розрахунки **4**

4 Аналіз ситуацій порушення безпеки **6**

4.1 Ситуація 1: Перехоплення мережевого трафіку 6

4.2 Ситуація 2: Компрометація бази даних продавця 7

4.3 Ситуація 3: Повторне виконання операції 8

4.4 Ситуація 4: Модифікація параметрів платежу 8

4.5 Ситуація 5: Витік облікових даних користувачів 9

5 Банківський переказ через Open Banking **10**

6 Історичний криптографічний протокол: MicroMint **12**

7 Висновки **14**

Список літературних і не дуже джерел **16**

1 Мета

Метою даної лабораторної роботи є дослідження принципів побудови сучасних електронних платіжних систем, криптографічних механізмів, що забезпечують захист платіжної операції на кожному з її етапів та основних загроз, що виникають під час здійснення електронних платежів.

Для досягнення цієї мети було поставлено такі завдання:

1. Розглянути основні механізми безпеки електронних карткових платежів (TLS, токенізація, CVV/CVC, 3-D Secure, strong client authentication, цифрові підписи та HMAC, PCI DSS) з використанням чинних стандартів та технічної документації;
2. Побудувати та проаналізувати поетапно спрощену схему онлайн-платежу карткою;
3. Проаналізувати п'ять типових ситуацій порушення безпеки разом із відповідними заходами протидії для кожної з них;
4. Дослідити сучасні спосіб оплати банківський переказ, ініційований через Open Banking;
5. Розглянути історичну схему мікроплатежів MicroMint та провести її порівняння з сучасними механізмами безпеки платежів.

2 Механізми безпеки сучасних електронних платежів

Питання 1. Учасники онлайн-платежу за допомогою картки та їхні ролі. У платіжній операції за допомогою картки завжди беруть участь однакові сторони, навіть якщо клієнт бачить лише сторінку оформлення замовлення продавця [1]:

- **Клієнт (cardholder)** – володіє карткою та рахунком, з якого знімаються кошти. Клієнт вводить реквізити картки (або обирає збережену картку / електронний гаманець) і, якщо це потрібно, підтверджує платіж.
- **Продавець (merchant)** – інтернет-магазин, що продає товари або послуги. Він створює замовлення, надсилає запит на оплату своєму платіжному провайдеру та, зрештою, отримує гроші.
- **Платіжний провайдер (PSP, Payment Service Provider) / acquirer** – еквайєр (банк-еквайєр) обробляє карткові платежі від імені продавця та направляє їх до карткової мережі; PSP або платіжний процесор (наприклад, Stripe) є технічним посередником, який часто поєднує обидві функції [1].
- **Платіжна система (card network)** – Visa, Mastercard тощо. Вона направляє запит на авторизацію до відповідного емітента за номером картки, а згодом здійснює переказ коштів між банками під час клірингу та розрахунків [1].
- **Емітент (issuing bank)** –банк, який випустив картку. Він здійснює аутентифікацію власника картки (наприклад, через 3-D Secure), перевіряє наявність коштів, ліміти та ознаки шахрайства, а також *схвалює або відхиляє* транзакцію [1].

Приклад: Клієнт купує книгу за 600 грн. Магазин (торговець) користується послугами PSP; еквайєр PSP надсилає запит через Visa (мережу) до банку клієнта (емітента), який резервує 600 грн і повертає код схвалення.

Питання 2. Ідентифікація користувача vs авторизації платежу. Ідентифікація (Authentication) відповідає на питання «*Хто ви?*» – система перевіряє, чи дійсно особа є законним власником картки або рахунку, наприклад, за допомогою пароля, одноразового коду або відбитка пальця [2]. **Авторизація (Authorisation)** платежу відповідає на запитання «*Чи дозволена ця конкретна операція?*» – емітент перевіряє, чи картка дійсна та не заблокована, чи є достатньо коштів або кредиту, і чи збігаються дані; потім він надсилає код схвалення або відмови та резервує суму платежу [1].

Аутентифікація стосується *особи*, авторизація – *транзакції*. Користувачеві, який пройшов повну аутентифікацію, все одно може бути відмовлено (через недостатність коштів), а платіж може бути

авторизовано без етапу посиленої аутентифікації, якщо застосовується виняток (наприклад, платіж на невелику суму, більше про це в питані №7).

Питання 3. Роль протоколу TLS (Transport Layer Security) в електронному платежі. TLS створює захищений канал між двома кінцевими точками – браузером клієнта та інтернет-магазином, інтернет-магазином та PSP, PSP та еквайром. Його поточна версія, TLS 1.3, була опублікована в липні 2026 року як RFC 9846, що замінює RFC 8446 [3]. TLS забезпечує три властивості [3]:

- **Автентифікація сервера** – браузер перевіряє сертифікат сервера, тому зловмисник не може видати себе за інтернет-магазин (захист від атак типу MiTM та фішингу на мережевому рівні, за умови, що користувач перебуває на правильному домені);
- **Конфіденційність** – дані картки, файли cookie та деталі замовлення шифруються, тому пасивне перехоплення інформації є марним;
- **Цілісність** – кожен запис захищений за допомогою AEAD (Authenticated Encryption with Associated Data), тому будь-які зміни під час передачі виявляються.

TLS *не приховує* обсяг даних або сам факт зв'язку [3] і захищає дані *лише під час передачі*: щойно дані досягають сервера, TLS більше не допомагає (див. розділ 3).

Питання 4. Токенізація платіжних даних. Токенізація замінює конфіденційний номер картки – PAN (Primary Account Number) – на substitute value, тобто платіжний токен. Відповідність "токен $\leftrightarrow$ PAN" зберігається лише в захищеному сховищі токенів постачальника токенів (TSP) і може бути відновлена лише уповноваженими суб'єктами [4]. Платіжний токен EMV додатково прив'язаний за допомогою *Token Domain Restriction Controls* до конкретного продавця, пристрою або каналу, тому токен, викрадений в одному магазині, не може бути використаний деінде [5]. Переваги зберігання токена замість PAN:

- У разі витоку даних отримані дані є марними поза межами їхньої домени [5];
- Втрачений пристрій або скомпрометований токен можна *від'єднати* без перевипуску фізичної картки [4];
- Торговець не зберігає номери PAN, що суттєво зменшує сферу застосування стандарту PCI DSS scope (Q9).

В Україні термін *card token* було додано до Положення НБУ про аутентифікацію у 2026 році: токен, створений після сильної аутентифікації емітентом, може сам по собі слугувати одним із елементів SCA («possession») [6]. *Приклад:* Apple Pay зберігає токен `4895 xxxx xxxx 1234` на телефоні; навіть якщо база даних магазину витокує його, токен працює лише для Apple Pay на цьому конкретному пристрої.

Питання 5. Призначення CVV/CVC та причини обмеження його зберігання CVV2/CVC2 (Card Verification Value / Card Validation Code) – це трьо/чотиризначний код захисту картки. Його запитують під час транзакцій без фізичної присутності картки (card-not-present, CNP), щоб підтвердити, що клієнт *фізично володіє* карткою, а не лише знає її номер [7]. Оскільки цей код не закодований на магнітній смузі чи в чіпі, викрадена база даних номерів карток без них стає набагато менш корисною. Саме тому стандарт PCI DSS класифікує цей код як *Sensitive Authentication Data (SAD)* і забороняє його зберігання після авторизації – навіть у зашифрованому вигляді та навіть за згодою клієнта [8, 9]. Якщо торговець зберігав би CVV-коди, одного витоку даних було б достатньо, щоб надати зловмисникам усе необхідне для здійснення онлайн-шахрайства.

Питання 6. Мета 3-D Secure та додаткової автентифікації користувача. 3-D Secure (3DS, «three-domain secure»: домен merchant/acquirer, домен взаємодії та домен емітента) – це протокол EMVCo, який дозволяє торговцю та емітенту обмінюватися даними з метою автентифікації власника картки

під час онлайн-покупки та запобігання шахрайству CNP [10]. Сервер 3DS продавця надсилає дані про транзакцію, платіж та пристрій через сервер каталогів карткової мережі на сервер контролю доступу (Access Control Server, ACS) емітента [11]. На основі цих даних емітент приймає рішення:

- **Безперешкодний потік (frictionless flow)** – низький ризик, клієнт просто натискає «Buy»;
- **Процес із перевіркою (challenge flow)** – вищий ризик, клієнт повинен підтвердити операцію за допомогою одноразового пароля, біометричних даних або іншим способом [10].

Результатом є криптографічне значення автентифікації, яке додається до запиту на авторизацію та підтверджує, що емітент автентифікував власника картки. Таким чином, 3DS є практичним способом реалізації двофакторної сильної автентифікації клієнта (strong customer authentication, SCA) для карткових платежів [10].

Вимоги до сильної автентифікації в Україні (Постанова НБУ № 58) *Положення про автентифікацію та застосування сильної автентифікації на платіжному ринку* (Постанова НБУ № 58 від 3 травня 2023 року, із змінами, внесеними Постановою № 4 від 13 січня 2026 року) встановлює такі основні правила [12, 6]:

- **Означення.** Посилена автентифікація використовує щонайменше два елементи з *різних* категорій: знання (те, що знає лише користувач), володіння (предмет, який має лише користувач) та властивість (біометричні дані) [12, п. 3(15)]. Компрометація одного елемента не повинна становити загрози для інших [12, п. 32].
- **Коли це є обов'язковим.** Для віддаленого доступу до облікового запису, для ініціювання віддаленого платежу та для будь-якої дії, щодо якої є підозра у шахрайстві [12, п. 7, 15].
- **Код автентифікації.** Результатом SCA є код автентифікації, який повинен бути захищеним від підробки, не повинен розкривати жодних елементів і не повинен бути виведеним із попередніх кодів; він реалізується за допомогою одноразових паролів, (кваліфікованих) електронних підписів або інших криптографічних засобів [12, п. 14, 16], [6].
- **Динамічне зв'язування (linking).** Платник бачить суму та отримувача; код прив'язаний саме до цієї суми та отримувача, і будь-яка зміна цих даних робить код недійсним [12, п. 22].
- **Безпека сеансу.** Не більше п'яти послідовних невдалих спроб, після чого аутентифікація блокується; не більше десяти хвилин бездіяльності після входу в систему [12, п. 17].
- **Винятки (Exemptions).** SCA можна пропустити, наприклад, для дистанційних платежів на суму до 2 000 UAH (з сукупними лімітами 10 000 UAH або п'ятьма послідовними операціями), для довірених одержувачів, для наступних платежів у рамках серії регулярних платежів, для переказів між власними рахунками та для операцій з низьким рівнем ризику згідно з аналізу ризиків транзакцій (transaction risk analysis, TRA) з орієнтовними показниками шахрайства для порогових значень 2 000, 7 000 і 20 000 UAH [12, п. 37–47, Додаток]. З 2026 року також звільняються від SCA податки та комунальні платежі на суму до 30 000 UAH [6].

Приклад: Для переказу 5 000 UAH новому одержувачу в банківському додатку потрібно, наприклад, ввести PIN-код (knowledge) на зареєстрованому смартфоні (knowledge), а на екрані підтвердження відображається точна сума та одержувач.

Питання 8. Цифрові підписи та HMAC між компонентами платіжної системи. Протокол TLS захищає повідомлення лише між двома сусідніми вузлами. Платіжні повідомлення часто проходять через кілька систем і реєструються, ставляться в чергу або пересилаються, тому їхня цілісність та походження мають бути перевірені на всьому шляху (*end-to-end*):

- **HMAC (Hash-based Message Authentication Code)** обчислює тег на основі повідомлення та спільного секретного ключа за допомогою геш-функції[13]. Це доводить, що повідомлення було створено особою, яка знає ключ, і не зазнало змін. *Приклад:* Stripe підписує кожен вебхук (наприклад, «платіж успішно здійснено») за допомогою HMAC-SHA256 на основі `timestamp.body`; магазин перераховує тег і відхиляє підроблені або застарілі сповіщення [14].

- **Digital signature** використовує приватний ключ для підписання та відкритий ключ для перевірки. Окрім цілісності, він забезпечує незаперечність (non-repudiation): підписувач не може згодом заперечити факт надсилання повідомлення, оскільки ніхто інший не володіє приватним ключем. НБУ вимагає використання електронних підписів, унікальних ідентифікаторів сеансу та міток часу за UTC у електронній взаємодії між учасниками платіжного ринку [12, п. 74–75], а в рамках Open Banking банк може вимагати від сервісу ініціювання платежів завіряти свої повідомлення кваліфікованою електронною печаткою [15, п. 30].

Мета стандарту PCI DSS та що він захищає. PCI DSS (Payment Card Industry Data Security Standard), який підтримується Радою з питань стандартів безпеки PCI, є набором базових технічних та операційних вимог, призначених для захисту даних платіжних рахунків [16]. Поточна версія – v4.0.1. Він захищає дві групи *account data*:

- **Дані власника картки (Cardholder Data, CHD)** – номер платіжної картки (PAN), ім'я власника картки, термін дії, код послуги; їх можна зберігати лише за умови захисту (наприклад, PAN має бути зроблений нечитабельним шляхом шифрування, обрізання або токенізації);
- **Чутливі дані автентифікації (Sensitive Authentication Data, SAD)** – повні дані магнітної смуги, коди перевірки картки та PIN-коди/блоки PIN-кодів; їх ніколи не можна зберігати після авторизації [8].

Стандарт застосовується до всіх суб'єктів, які зберігають, обробляють або передають CHD/SAD або можуть впливати на безпеку середовища даних власника картки (Cardholder Data Environment, CDE): торговців, процесорів, еквайрів, емітентів та постачальників послуг [16]. Він охоплює всю інфраструктуру: засоби контролю мережевої безпеки, безпечну конфігурацію, шифрування даних у стані спокою та під час передачі, управління вразливостями, контроль доступу та багатофакторна автентифікація (Multi-Factor Authentication, MFA), ведення журналів та моніторинг, регулярне тестування та політики безпеки.

3 Карткові онлайн-розрахунки

Розглянемо клієнта, який купує товари в інтернет-магазині та розраховується банківською карткою. Магазин використовує сучасного PSP: поля для введення даних картки на сторінці оформлення замовлення обслуговуються безпосередньо PSP (hosted fields), тому власний сервер магазину ніколи не бачить повного номера картки. На рисунку 1 показано спрощений алгоритм дій.

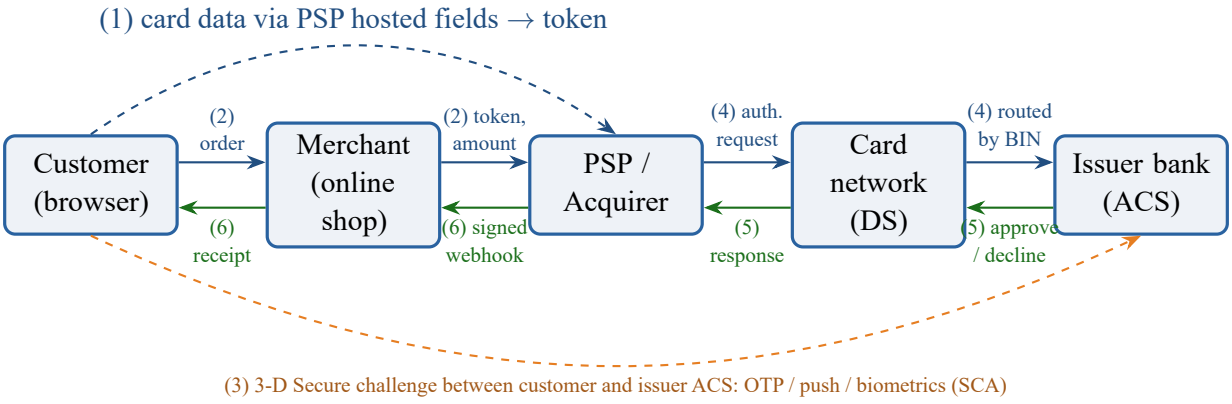


Рис. 1: Спрощена схема карткового онлайн-платежу з токенізацією та 3-D Secure.

Основні етапи: (1) клієнт вводить дані картки у поля, розміщені на сервері PSP, та отримує токен; (2) замовлення та токен надсилаються торговцю, який звертається до PSP із запитом на списання

коштів; (3) за допомогою 3-D Secure здійснюється автентифікація власника картки в емітента; (4) запит на авторизацію надходить через еквайра та платіжну мережу до емітента; (5) емітент схвалює або відхиляє запит; (6) результат повертається торговцю у вигляді відповіді API та підписаного вебхука. Пізніше в процесі клірингу та розрахунків фактичні кошти перераховуються від емітента до еквайра та продавця [1]. У таблиці 1 нижче наведено підсумкові дані щодо інформації та захисту на кожному етапі.

Таблиця 1: Дані, доступ та захист на основних етапах онлайн-платежу карткою.

Етап	**Передані дані**	**Хто має доступ**	**Що необхідно захищати**	**Механізми**
1. Введення даних картки	PAN, термін дії, CVV2, ім'я власника картки	Клієнт, PSP (сервер торговця — ні)	Усі поля: конфіденційність; CVV2 не повинен зберігатися	TLS 1.3, поля, розміщені на сервері PSP / iframe, негайна токенізація, PCI DSS [16]
2. Оформлення замовлення	Ідентифікатор замовлення, сума, валюта, токен, ключ ідемпотентності	Клієнт, торговець, PSP	Сума та одержувач платежу: цілісність; токен: конфіденційність	TLS, обчислення суми на стороні сервера, секретний ключ API, ключ ідемпотентності [17]
3. 3-D Secure	Дані про транзакції та пристрої, одноразовий пароль (OTP) / підтвердження через push-повідомлення, значення автентифікації	Сервер 3DS торговця, сервер каталогів, ACS емітента, клієнт	Одноразовий пароль (OTP) та біометричні дані: конфіденційність; сума та одержувач платежу, що відображаються користувачеві: цілісність	TLS на всіх етапах, SCA з динамічним зв'язуванням, криптографічна автентифікація значення [10, 12]
4. Запит на авторизацію	PAN або мережевий токен, сума, ідентифікатор торговця, значення автентифікації, криптограма	Еквайєр, платіжна мережа, емітент	PAN: конфіденційність; сума, торговець: цілісність	Зашифровані приватні мережі, MAC-код у повідомленнях, HSM (апаратний модуль безпеки) для ключів, зіставлення токенів із PAN у сховищі [4]
5. Рішення	Код схвалення / відмови, код авторизації	Емітент, мережа, еквайєр, PSP	Цілісність відповіді (підробка статусу «затверджено» має бути неможливою)	Автентифіковані канали, MAC, зіставлення за ідентифікатором транзакції

Continued on next page

Таблиця 1: Дані, доступ та захист на основних етапах онлайн-платежу карткою. (Continued)

Етап	Передані дані	Хто має доступ	Що необхідно захищати	Механізми
6. Повідом-лення	Статус платежу, ідентифікатор замовлення, сума	ПСП, торговець	Цілісність та автентичність статусу	Підпис HMAC-SHA256 підпис із позначкою часу на веб-хуках [14]

Де і як відбувається автентифікація користувача. Автентифікацію клієнта здійснює *issuer*, а не магазин: під час процедури 3-D Secure система ACS емітента або автентифікує клієнта у фоновому режимі на основі даних про ризики (без додаткових дій, frictionless), або вимагає підтвердження – зазвичай у вигляді push-повідомлення в банківському додатку з введенням PIN-коду або відбитка пальця, що є двофакторною SCA [10, 12]. Інтернет-магазин отримує лише результат, але ніколи не дізнається банківські реквізити користувача.

Як операція схвалюється або відхиляється. Остаточне рішення приймає емітент під час авторизації: він перевіряє стан картки (чи не загублена або заблокована), наявність коштів або кредитного ліміту, результат 3DS та власний рейтинг ризику шахрайства. Він повертає код відповіді; у разі схвалення сума резервується на рахунку та видається код авторизації [1]. Продавець відправляє товар лише після отримання підтвердженого статусу «успішно».

Чому одного TLS недостатньо

TLS є необхідним, але охоплює лише один клас загроз – атаки на дані *під час передачі (in transit)* між двома сусідніми вузлами. Він не вирішує таких проблем:

- **Дані у стані спокою (Data at rest)** Після розшифрування на сервері дані перебувають у вигляді відкритого тексту; злам бази даних, дії інсайдера або шкідливе програмне забезпечення на сервері не підпадають під дію TLS. Цю проблему вирішують токенізація та стандарт PCI DSS.
- **Ендпоїни (Endpoints)** Шкідливе програмне забезпечення (Malware) в браузері клієнта (наприклад, скімер, вбудований у сторінку оформлення замовлення) зчитує дані *ще до* їх шифрування.
- **Ідентичність платника** TLS перевіряє автентичність *сервера*, а не власника картки. Злодій із викраденими даними картки може встановити цілком дійсний сеанс TLS; тут необхідні SCA/3-D Secure.
- **Фішинг.** Фальшивий інтернет-магазин може мати дійсний сертифікат для власного домену, схожого на справжній.
- **Бізнес-логіка** Повторне відтворення правильного запиту, подвійне списання коштів або зміна суми, спричинені легітимним, але скомпрометованим компонентом, не виявляються за допомогою TLS; необхідні ідемпотентні ключі, нонси, HMAC/підписи та динамічне зв'язування.
- **Ланцюги з декількома проміжними вузлами (Multi-hop chains)** TLS працює "від вузла до вузла"(hop-by-hop): платіжний сервіс-провайдер (PSP), мережа та емітент по черзі завершують сеанс, тому для забезпечення цілісності "від кінця до кінця" та неможливості заперечення авторства потрібні підписи на рівні повідомлень.

4 Аналіз ситуацій порушення безпеки

4.1 Ситуація 1: Перехоплення мережевого трафіку

Зловмисник може спостерігати за трафіком між клієнтом та магазином; з'єднання правильно використовує HTTPS/TLS (Малюнок 2).

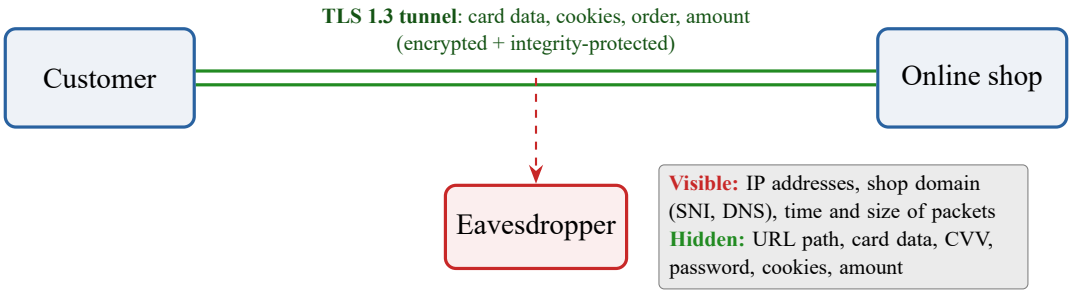


Рис. 2: Situation 1: passive observation of a correctly configured TLS connection.

Що може дізнатися зловмисник. Лише метадані: хто з ким спілкується (IP-адреси), ім'я домену (зазвичай видно з Server Name Indication та DNS-запитів), коли та скільки даних надсилається. TLS явно не приховує довжину повідомлень [3], тому аналіз трафіку може виявити, наприклад, чи була завантажена сторінка оплати.

Від чого захищає TLS. Зчитування вмісту (дані картки, CVV, логін, сесійні файли cookie), модифікацію або injecting пакетів (будь-яка зміна порушує тег AEAD), видавання себе за інтернет-магазин (перевірка сертифіката завершиться невдачею) та повторне відтворення записаних записів (послідовні номери та свіжі сесійні ключі). Оскільки TLS 1.3 використовує ефемерний алгоритм Діффі–Хелмана, записаний трафік залишається захищеним навіть у випадку подальшого витоку довгострокового ключа сервера (forward secrecy) [3].

Наслідки та залишкові ризики. При правильному використанні TLS наслідки обмежуються витоком метаданих, що стосуються конфіденційності. Ризик зростає лише у разі неправильної конфігурації TLS (застарілі версії, вимкнена перевірка сертифікатів) або якщо користувач приймає попередження про сертифікат. Заходи протидії: виключно TLS 1.3, HSTS (HTTP Strict Transport Security) для запобігання переходу на HTTP, правильна перевірка сертифікатів у мобільних додатках.

4.2 Ситуація 2: Компрометація бази даних продавця

Зловмисник отримав копію бази даних інтернет-магазину. На рис. 3 порівняно дві стратегії зберігання даних.

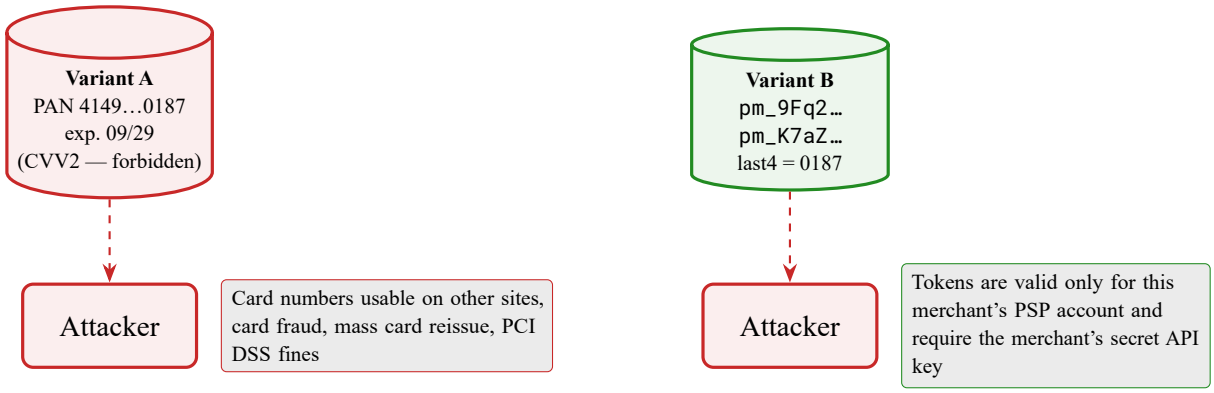


Рис. 3: Situation 2: database leak with stored card data (A) vs. stored tokens (B).

Варіант A – магазин зберігав дані карток. Зловмисник отримує номери карток (PAN), дати закінчення терміну їх дії, а також імена всіх клієнтів. Ці дані можна продати або використати в інших інтернет-магазинах (особливо тих, що мають слабкі перевірки 3DS). Якщо магазин також незаконно зберігав коди CVV, витік стає повним "набором" для шахрайства CNP [9]. Наслідки: прямі збитки для клієнтів, масове переоформлення карток банками, штрафи та можлива втрата права приймати картки продавцем, втрата репутації.

Варіант Б — магазин зберігав лише платіжні токени. Токени є сурогатними значеннями. Вони обмежені доменом: токен може використовуватися лише цим торговцем через його PSP, а PSP додатково вимагає секретного API-ключа торговця [5]. За межами цієї доменної зони токен не має вартості, і постачальник токенів (TSP) може анулювати або відв'язати токени без перевипуску карток [4]. Витік все одно розкриває особисті дані (імена, адреси, останні чотири цифри), але не дані карток, придатні для використання.

Роль токенізації. Токенізація переміщує цінні дані (PAN) із системи продавця у спеціалізоване, захищене сховище токенів. Таким чином вона зменшує радіус ураження (blast radius) порушення та обсяг вимог PCI DSS для продавця. Це необхідно поєднувати із захистом ключів API: якщо зловмисник викраде базу даних і секретний ключ, він зможе ініціювати списання коштів за допомогою токенів – тому ключі мають зберігатися в окремому сховищі секретних даних, а не в тій самій базі даних.

4.3 Ситуація 3: Повторне виконання операції

Зловмисник отримав правильний запит на оплату та повторно надсилає його до платіжної системи (рис. 4).

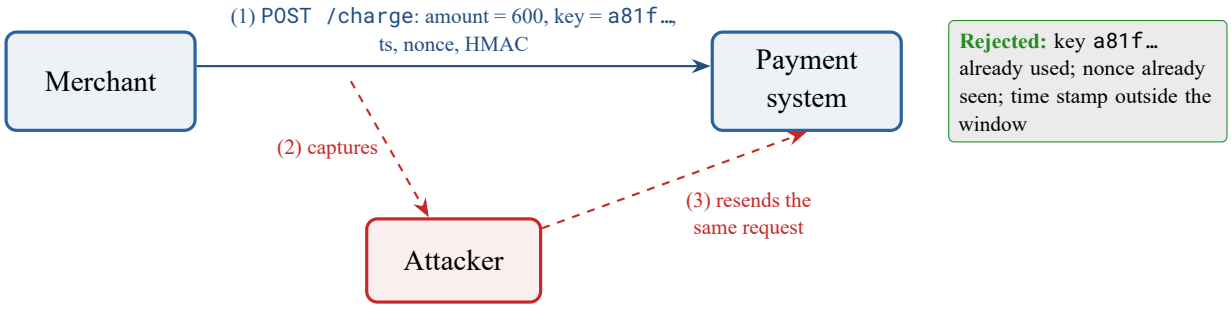


Рис. 4: Situation 3: replay attack and its detection.

Тип атаки. Це атака *повторного відтворення* (replay attack): зловмиснику не потрібно розшифровувати чи модифікувати повідомлення достатньо лише повторно надіслати дійсне повідомлення. Навіть запит із дійсним підписом приймається, оскільки сам підпис залишається правильним. Наслідок: з клієнта стягується плата двічі або більше.

Контрзаходи.

- **Ключ ідемпотентності (Idempotency key)** – унікальне випадкове значення (наприклад, UUID v4) для кожної операції. Сервер зберігає результат першого запиту з цим ключем і повертає той самий результат у разі повторення, тому платіж ніколи не виконуватиметься двічі [17].
- **Nonce та мітка часу всередині signed/HMAC-protected part** – одержувач відхиляє повідомлення, старші за короткий проміжок часу (Stripe використовує приблизно п'ять хвилин для веб-хуків), та запам'ятовує нещодавно отримані nonce значення [14].
- **Унікальні ідентифікатори транзакцій та прив'язка до сеансу** – НБУ вимагає, щоб кожна електронна взаємодія містила унікальний ідентифікатор сеансу, номер операції та мітку часу за UTC [12, п. 74].
- **Одноразові коди автентифікації** – код 3DS або SCA дійсний лише для однієї операції, тому перехоплений код не може авторизувати інший платіж [12, п. 16].
- **Протокол TLS** з одного боку, запобігає перехопленню запиту в мережі, але не захищає від повторного відтворення запиту з боку скомпрометованого компонента, тому зазначені вище заходи на рівні додатка залишаються необхідними.

4.4 Ситуація 4: Модифікація параметрів платежу

Під час здійснення платежу хтось намагається змінити суму або отримувача (Figure 5).

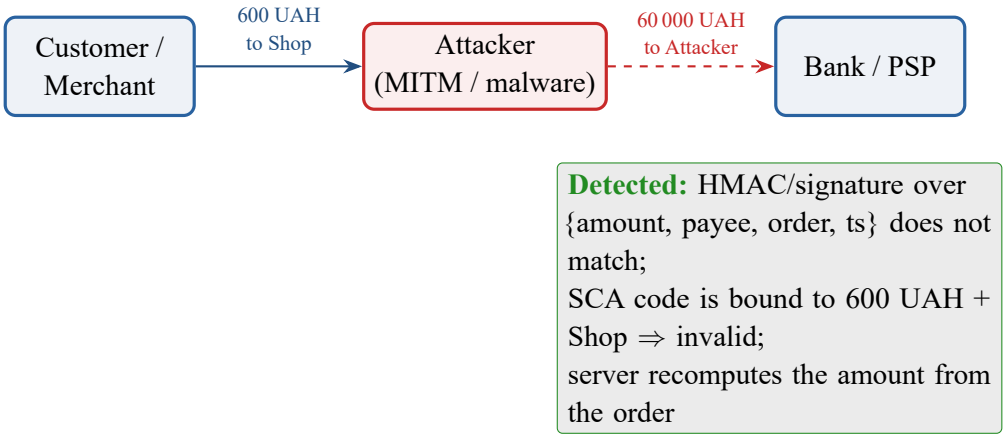


Рис. 5: Situation 4: attempt to modify the amount or payee.

Можливі наслідки. Клієнт сплачує більше, ніж було домовлено, або гроші надходять на рахунок зловмисника замість магазину. Такі атаки можуть здійснюватися за допомогою мережевої атаки man-in-the-middle, шкідливого програмного забезпечення в браузері (так званий "man-in-the-browser", що переписує форму), або шляхом фальсифікації параметрів на стороні клієнта (наприклад, зміни прихованого поля суми `amount`).

Криптографічні механізми захисту.

- **TLS** – будь-яка зміна даних під час передачі порушує тег автентифікації запису [3].
- **HMAC або цифровий підпис над критичними полями** (сума, валюта, одержувач, ідентифікатор замовлення, мітка часу) – одержувач перераховує тег; будь-яка зміна виявляється навіть після декількох проміжних вузлів [13].
- **Динамічне зв'язування (Dynamic linking) (НБУ №. 58)** – код автентифікації SCA обчислюється на основі суми та одержувача, що відображаються користувачеві; будь-яка зміна робить код недійсним, і операція відхиляється [12, п. 22]. Це має вирішальне значення для захисту від шкідливого програмного забезпечення: навіть якщо браузер заражений, користувач бачить справжню суму та одержувача в окремому банківському додатку.

Механізми на рівні додатків. Жодної довіри до суми, надісланої клієнтом – сервер перераховує її на основі запиту у своїй базі даних; перевірка одержувача за даними рахунку продавця; реквізити рахунку PSP та реквізити виплати можна змінити лише після додаткової автентифікації; ліміти та виявлення аномалій на стороні емітента; у системі Open Banking банку заборонено змінювати інструкцію, отриману від ініціатора, а ініціатору заборонено змінювати суму [15, п. 27, 35].

4.5 Ситуація 5: Витік облікових даних користувачів

Зловмисник має логін та пароль користувача платіжної системи, але кожна операція вимагає додаткового підтвердження (рис 6).

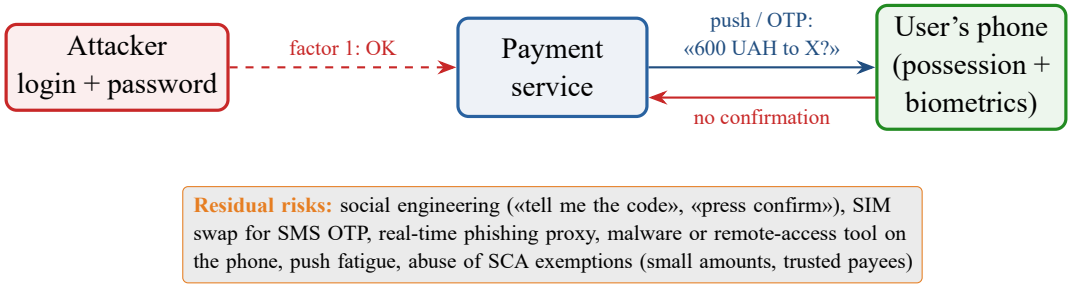


Рис. 6: Situation 5: stolen password vs. multi-factor authentication.

Роль MFA і 3-D Secure. Пароль є лише *knowledge factor*. Багатофакторна автентифікація (MFA) вимагає другого, незалежного фактора – володіння (*possession*) (зареєстрований телефон, апаратний ключ) або притаманності (*inherence*) (відбиток пальця, обличчя) [2, 12]. Зловмисник знає пароль, але не має телефону жертви, тому операцію неможливо підтвердити. Для карткових платежів 3-D Secure забезпечує саме таку перевірку з боку емітента [10], а НБУ обмежує кількість спроб введення даних – п'ятьма невдалими спробами [12, п. 17]. Крім того, завдяки динамічному зв'язку користувачеві відображаються реальна сума та отримувач платежу, тому несподіваний запит на підтвердження є очевидним сигналом тривоги.

Залишкові ризики. Багатофакторна автентифікація (MFA) суттєво знижує, але не усуває (!) ризик:

- **Соціальна інженерія (social engineering)** – користувача переконують продиктувати одноразовий пароль (one-time password, OTP) або підтвердити push-повідомлення (дзвінок від "служби безпеки банку");
- **Заміна SIM / перехоплення SMS** – SMS-коди прив'язані до номера телефону, а не до пристрою;
- **Фішинг у реальному часі (зловмисник-посередник)** – підроблений сайт негайно пересилає одноразовий пароль (OTP); запобігти цьому можуть лише захищені від фішингу засоби автентифікації (FIDO2 / ключі доступу), які прив'язують відповідь до справжнього домену [2];
- **Зкомпроментований пристрій** – шкідливе програмне забезпечення або програми віддаленого доступу на телефоні;
- **Винятки** – платежі на невеликі суми або на користь довірених одержувачів можуть здійснюватися без SCA, згідно до [12, п. 37–39], тому зловмисник може спробувати здійснити багато дрібних платежів або додати себе до списку довірених осіб (що саме по собі вимагає SCA [12, п. 38]);
- **Витік конфіденційної інформації** – навіть без здійснення платежу зловмисник може побачити інформацію про обліковий запис, якщо сам факт входу надає доступ до неї.

Тому MFA необхідно доповнити моніторингом шахрайства, повідомленнями про кожен вхід та платіж.

5 Банківський переказ через Open Banking

Open Banking це структурований та безпечний обмін даними між банком, що веде рахунок, та стороннім постачальником послуг через спеціалізовані інтерфейси (APIs), який здійснюється виключно за згодою користувача [15, п. 2]. В Україні це регулюється Постановою НБУ № 80 (набула чинності 1 серпня 2025 року) разом із Постановою № 58 [15, 12]. Замість введення реквізитів картки клієнт здійснює оплату *безпосередньо з банківського рахунку* шляхом кредитного переказу, який ініціює сторонній додаток, а клієнт підтверджує у своєму банку. Українські банки базували специфікацію своїх API на європейському стандарті NextGenPSD2 від Berlin Group [18]; на міжнародному рівні найпоширенішим профілем безпеки для таких APIs є OpenID Foundation's FAPI (Financial-grade API) [19].

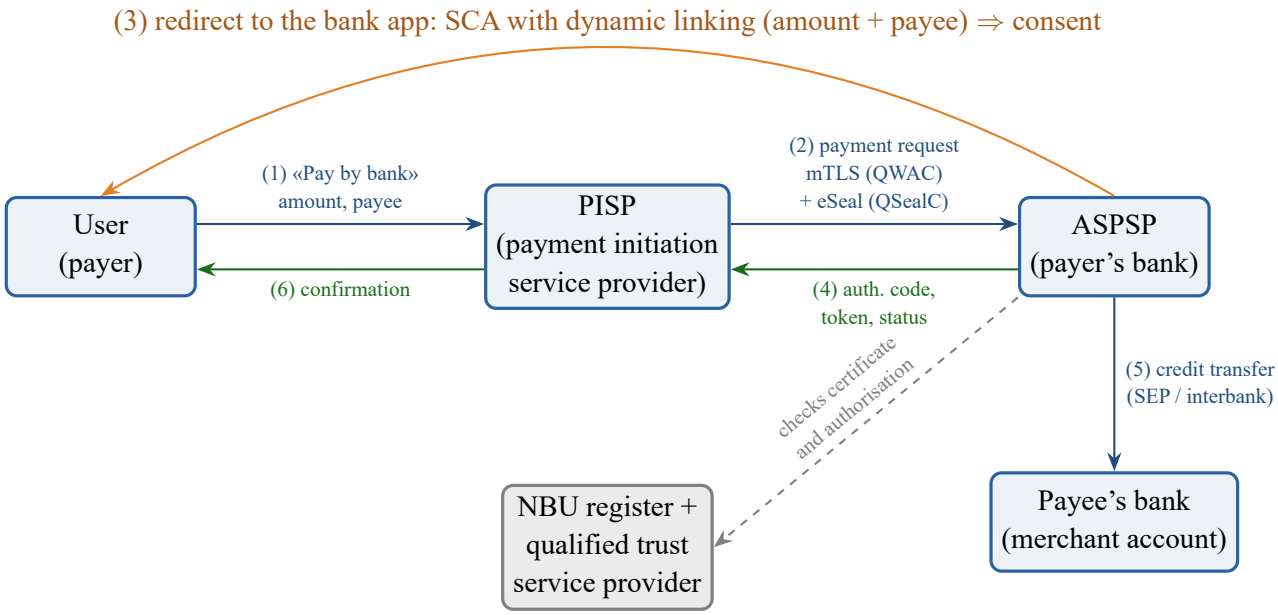


Рис. 7: Спрощена схема платежу, ініційованого через Open Banking

Основні учасники [15, п. 3, 6—8]:

- **Платник (payer)** – власник рахунку, який надає згоду;
- **ASPSP (Account Servicing Payment Service Provider)** – постачальник платіжних послуг, що обслуговує рахунок. Фактично це банк, у якому відкрито рахунок; він здійснює автентифікацію користувача та виконує платіж;
- **PISP (Payment Initiation Service Provider)** – постачальник послуг ініціювання платежів, третя сторона (додаток, каса магазину або інший банк), уповноважена НБУ ініціювати платежі; ПІС (постачальники послуг інформації про рахунки, або англійською – AISPs (Account Information Service Providers)) лише зчитують залишки та історію;
- **Банк одержувача** – отримує грошовий переказ; НБУ веде реєстр уповноважених постачальників послуг.
- **Технічний оператор** (optional) – забезпечує технічне з'єднання;

Дані та криптографічні облікові дані. Платіжна інструкція (сума, IBAN отримувача, ім'я, призначення платежу); кваліфіковані сертифікати PISP для відкритого банкінгу – сертифікат автентифікації веб-сайту (QWAC), що використовується у взаємному TLS, та сертифікат електронної печатки (QSealC), який застосовується для завірення платіжного повідомлення [15, п. 16, 30—31, 34]; коди авторизації типу OAuth та короткочасні токени доступу, прив'язані до сертифіката клієнта [19, 20]. Власні облікові дані користувача (PIN-код, біометричні дані, ключі пристрою) використовуються *виключно* всередині додатка банку.

Де зберігається найважливіша інформація. Сам рахунок, елементи автентифікації користувача та ключі SCA залишаються в ASPSP. PISP заборонено зберігати конфіденційні платіжні дані користувача, утримувати кошти користувача або змінювати параметри платежу [15, п. 35]; він повинен забезпечувати безпеку своїх власних приватних ключів (для mTLS та електронної печатки(eSeal)), зазвичай у HSM.

Як відбувається автентифікація користувача. Згоду на кожен платіж платник надає банку через власний додаток банку з використанням своєї автентифікації, яку повинен здійснювати ASPSP [15, п. 47–48]. PISP ніколи не бачить банківський пароль користувача — це головна перевага з точки зору безпеки порівняно зі screen scraping або введенням реквізитів картки.

Як підтверджується операція. Код автентифікації SCA динамічно пов'язаний із сумою та одержувачем [12, п. 22]. Після успішного проходження SCA банк видає код авторизації, який PISP обмінює на токен, прив'язаний до відправника; потім банк виконує кредитний переказ, не має права змінювати інструкцію та повідомляє PISP про статус [15, п. 27–28].

Криптографічні механізми.

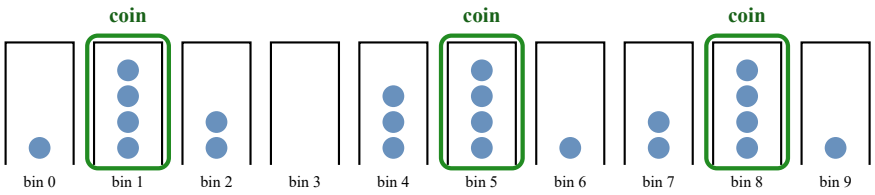
- **Взаємний (mutual) TLS** із кваліфікованими сертифікатами: банк аутентифікує PISP та перевіряє його у реєстрі НБУ перед кожним доступом [15, п. 16];
- **Кваліфікована/розширена електронна печатка** на платіжному повідомленні PISP підтверджує цілісність та невідмовність авторства інструкції [15, п. 30];
- **SCA з динамічним зв'язуванням** (одноразові коди або підписи в банківському додатку) [12];
- **Зміцнення безпеки OAuth 2.0** як у FAPI 2.0: запити на авторизацію, що надсилаються з боку сервера (pushed authorisation requests, PAR), щоб параметри запиту не можна було змінити в браузері [21], PKCE, коди авторизації, дійсні не більше 60 секунд, та токени з обмеженням відправника через mTLS або DPoP, щоб викрадений токен не міг бути використаний іншим клієнтом [19].

Типові загрози. Фішинг, що імітує перенаправлення на офіційний сайт банку – користувача переконують підтвердити платіж, ініційований шахраєм (шахрайство з авторизованими push-платежами); зловмисний або скомпрометований PISP або крадіжка його приватних ключів; перехоплення або витік кодів авторизації та токенів; маніпулювання параметрами платежу до SCA; атаки на доступність API (НБУ визначає збій як п'ять запитів, на які не отримано відповіді протягом 30 секунд [12, п. 84]); а також надмірний збір даних третіми сторонами. Оскільки кредитний переказ зазвичай є незворотним (на відміну від карткового платежу, тут немає можливості повернення коштів), ретельна перевірка користувачем одержувача на екрані SCA має вирішальне значення.

6 Історичний криптографічний протокол: MicroMint

Проблема, що вирішується. MicroMint був запропонований Рональдом Рівестом та Аді Шаміром у 1996 році разом зі схемою PayWord для здійснення *мікроплатежів* – наприклад, оплата одним центом перегляду однієї веб-сторінки [22]. У разі карткових платежів комісія за обробку перевищує суму такого платежу, а операції з відкритими ключами були дорогими: типова робоча станція того часу могла виконувати приблизно два підписи RSA на секунду, але 20 000 геш-обчислень на секунду [22]. Метою MicroMint було повністю відмовитися від криптографії з відкритими ключами та звести вартість кожного платежу до кількох обчислень геш-функції, погодившись на нижчий, але *достатньо хороший* рівень безпеки – подібно до автоматів з цукерками, що працюють на монети, які не мають дорогих детекторів підробок і все одно функціонують [22].

Основна ідея: монета це колізія геш-функції Учасниками є *broker* (a bank-like mint), *users* та *vendors*. Монета – це набір із k різних значень $x_1, \ldots, x_k$, які дають однаковий геш: $h(x_1) = h(x_2) = \ldots = h(x_k)$, де k-кратна колізія [22]. Створення таких колізій схоже на випадкове кидання кульок у контейнери: монета – це контейнер, до якого потрапило k кульок (Figure 8). Пошук першої колізії є дуже витратним, але після досягнення цього порогу кількість колізій швидко зростає ($\approx c^k$ монет за c разів більший обсяг роботи). Тому брокер, що володіє потужним спеціалізованим обладнанням, виробляє монети дешево, тоді як дрібний фальшивомонетник платить за кожну монету набагато більше, ніж вона коштує [22].



Ball x goes into bin $y = h(x)$. A bin with $k = 4$ balls gives a coin (x_1, x_2, x_3, x_4). The vendor checks a coin with only 4 hash computations; the broker needs $\approx 2^{54}$ hashes per month to mint 2^{30} coins.

Рис. 8: MicroMint: minting coins as k-way hash collisions («balls into bins»).

Спрощений алгоритм роботи:

1. На початку місяця брокер оприлюднює критерій дійсності монет (геш-функцію та необхідні старші біти ”правильного” значення гешу). Монети дійсні лише протягом цього місяця [22].
2. Користувачі купують монети у брокера (наприклад, 2 500 монет за 25$), а брокер фіксує, які монети дісталися якому користувачеві.
3. Щоб розрахуватися, користувач передає продавцю невикористану монету. Продавець перевіряє її локально: обчислює k гешів і перевіряє, чи вони рівні та ”правильні” – зв’язок із брокером не потрібен.
4. Наприкінці дня продавець повертає зібрані монети брокеру, який оплачує кожну монету не більше ніж один раз і виявляє монети, які були витрачені двічі.

Посередник обирає $k = 4$, інвестує близько \$150 000 доларів у мікросхеми FPGA та диски й випускає 2^{30} (близько одного мільярда) монет на місяць, тоді як підробка однієї монети на звичайній робочій станції зайняла б приблизно 80 років [22].

Криптографічні механізми. Використовується лише одностороння геш-функція, стійка до колізій (автори застосовували гешування на основі DES або MD5/SHA, скорочене до короткого результату). Опціональні розширення передбачають використання *hidden predicates* (таємні щоденні умови щодо вартості монет, які розкриваються пізніше) та монет, *user-specific* або *vendor-specific*, де структура монети залежить від гешу ідентичності користувача або продавця [22].

Властивості безпеки.

- **Неможливість підробки з економічних міркувань** дрібна підробка коштує дорожче, ніж приносить прибутку; масштабна підробка виявляється, коли з’являються монети з «unused» контейнерів, і посередник може відкликати серію [22];
- **Обмежений термін дії** – монети втрачають чинність щомісяця, тому фальсифікатор завжди починає з запізненням;
- **double-spending detection** *після* взаємодії з брокером, який ідентифікує постійних шахраїв та виключає їх (незначні шахрайські дії допускаються);
- **Захист від крадіжки.** Монети, прив’язані до конкретного користувача чи продавця, є марними для інших користувачів та вендорів;
- **Відсутність анонімності.** Брокер знає, який користувач купив які монети; MicroMint не є цифровими грошима.

Comparison with modern payment mechanisms.

Таблиця 2: MicroMint у порівнянні з сучасними механізмами електронних платежів.

Аспект	**MicroMint (1996)**	**Сучасні платежі (2026)**
Основа безпеки	Економіка колізій хеш-функцій, «достатньо хороший» рівень безпеки	Надійна криптографія: TLS 1.3, HMAC, підписи, ключі, захищені HSM [3, 13]
Перевірка	Офлайн у продавця (4 хеші)	Онлайн-авторизація емітентом для кожного платежу [1]
Подвійна витрата	Виявляється пізніше; допускаються незначні втрати	Запобігається в режимі онлайн (резерв балансу, ідемпотентні ключі) [17]
Прив'язка вартості	Монети, прив'язані до користувача/продавця	Платіжні токени з обмеженням домену (продавець, пристрій, канал) [5]
Обмежений термін дії	Монети втрачають чинність щомісяця	Життєвий цикл токенів, одноразові коди SCA, 60-секундні коди авторизації [12, 19]
Підхід, заснований на оцінці ризиків	«Не переймайся дрібницями»	Винятки з SCA для платежів з низькою вартістю та низьким рівнем ризику (TRA) [12]
Автентифікація користувача	Не застосовується до окремих платежів (володіння монетами)	Обов'язкова SCA з динамічним зв'язуванням [12, 10]

Основна ідея MicroMint – *зробити кожну перевірку безпеки пропорційною до вартості, що піддається ризику, та використовувати дешеві симетричні примітиви замість операцій з відкритим ключем* де це можливо. Це актуально і зараз. Сучасні системи використовують HMAC замість підписів для повідомлень з великим обсягом даних (веб-хуки), прив'язують токени до домену, як і монети конкретних постачальників, та застосовують винятки на основі ризику для дрібних платежів. Різниця лише в тому, що сьогодні дешеві обчислення роблять надійну криптографію доступною для кожного платежу, а підключення до Інтернету дозволяє не просто виявляти подвійні витрати, а й запобігати їм. Сам MicroMint ніколи не набував широкого поширення: сьогодні питання мікроплатежів зазвичай вирішується шляхом агрегації (підписки, передплачені гаманці), а не за допомогою спеціальних монет.

7 Висновки

1. Онлайн-платіж карткою передбачає участь п'яти сторін – клієнта, продавця, PSP/еквайра, платіжної мережі та емітента. Аутентифікація (”хто платить?”) здійснюється емітентом за допомогою 3-D Secure та SCA, тоді як авторизація (”чи можна виконати цей платіж?”) – це рішення емітента, що ґрунтується на наявності коштів, статусі картки та оцінці ризику шахрайства.

2. Жоден окремий механізм не забезпечує повного захисту платежу. Протокол TLS захищає дані під час передачі; токенізація та стандарт PCI DSS захищають збережені дані; CVV та 3-D Secure підтверджують володіння карткою та особу її власника; HMAC та цифрові підписи гарантують цілісність та автентичність повідомлень на всіх етапах передачі; ідемпотентні ключі, нонси та часові мітки запобігають повторному відтворенню.

3. Аналіз п'яти ситуацій порушення безпеки показує, що сам по собі протокол TLS захищає лише від мережевих атак (Ситуація 1); витоки з баз даних запобігаються за допомогою токенізації (Ситуація 2), повторне відтворення – за допомогою ідемпотентності та одноразових значень

(Ситуація 3), модифікація за допомогою автентифікації повідомлень та динамічного зв'язування (Ситуація 4), а крадіжка облікових даних за допомогою багатофакторної автентифікації (Ситуація 5). Залишкові ризики, такі як соціальна інженерія та скомпрометовані пристрої, залишаються і вимагають моніторингу та навчання користувачів.

4. Українське законодавство (Постанови НБУ № 58 та № 4) визначає SCA відповідно до практики ЄС: два незалежні фактори, динамічне прив'язування коду до суми та отримувача, обмеження на кількість невдалих

5. Open Banking переносить найважливіші дані та аутентифікацію користувача до його власного банку: сторонній постачальник послуг проходить аутентифікацію за допомогою кваліфікованих сертифікатів, зашифровує свої повідомлення та ніколи не бачить облікових даних користувача. Головною загрозою, що залишається, є соціальна інженерія, оскільки кредитні перекази, як правило, є незворотними.

6. MicroMint демонструє одну з перших спроб збалансувати вартість безпеки та вартість платежу за допомогою використання виключно геш-функцій. Його основні ідеї це безпека пропорційна вартості, обмежений термін дії та прив'язка до користувача/продавця – знову з'являються у сучасній токенізації, короткострокових облікових даних та винятках із SCA.

Список літературних і не дуже джерел

[1] Stripe, Inc. *Credit Card Payment Authorization and Transaction Settlement Process*. 2023. (URL) (visited on 10/30/2026).

[2] David Temoshok et al. *NIST SP 800-63B-4 — Digital Identity Guidelines: Authentication and Authenticator Management*. 2025. (URL) (visited on 11/01/2026).

[3] Eric Rescorla. *RFC 9846 — The Transport Layer Security (TLS) Protocol Version 1.3*. Obsoletes RFC 8446. 2026. (URL) (visited on 10/31/2026).

[4] EMVCo. *EMV Payment Tokenisation*. 2026. (URL) (visited on 11/01/2026).

[5] EMVCo. *EMV Payment Tokenisation: Quick Reference Guide*. 2024. (URL) (visited on 11/01/2026).

[6] National Bank of Ukraine. *Resolution of the NBU Board No. 4 of 13 January 2026 "On Approval of Amendments to the Regulation on Authentication and Application of Strong Authentication in the Payment Market"*. In Ukrainian. 2026. (URL) (visited on 11/02/2026).

[7] PCI Security Standards Council. *FAQ: Are Merchants Allowed to Request Card Verification Codes/Values from Cardholders?* 2026. (URL) (visited on 11/02/2026).

[8] PCI Security Standards Council. *FAQ 1210: Storage of Sensitive Authentication Data after Authorization*. 2026. (URL) (visited on 11/02/2026).

[9] PCI Security Standards Council. *FAQ 1280: Can Card Verification Codes/Values Be Stored for Card-on-File or Recurring Transactions?* 2026. (URL) (visited on 11/02/2026).

[10] EMVCo. *EMV 3-D Secure*. 2026. (URL) (visited on 11/01/2026).

[11] EMVCo. *EMV 3-D Secure Whitepaper, Version 2.0: Business Overview, Technical Features and Use Cases*. 2025. (URL) (visited on 11/01/2026).

[12] National Bank of Ukraine. *Resolution of the NBU Board No. 58 of 3 May 2023 "On Approval of the Regulation on Authentication and Application of Strong Authentication in the Payment Market"*. In Ukrainian. 2023. (URL) (visited on 11/02/2026).

[13] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. *RFC 2104 — HMAC: Keyed-Hashing for Message Authentication*. 1997. (URL) (visited on 10/31/2026).

[14] Stripe, Inc. *Resolve Webhook Signature Verification Errors*. 2026. (URL) (visited on 10/31/2026).

[15] National Bank of Ukraine. *Resolution of the NBU Board No. 80 of 25 July 2025 "On Approval of the Regulation on Open Banking in Ukraine"*. In Ukrainian. 2025. (URL) (visited on 11/02/2026).

[16] PCI Security Standards Council. *PCI Data Security Standard (PCI DSS)*. 2026. (URL) (visited on 11/01/2026).

[17] Stripe, Inc. *Stripe API Reference: Idempotent Requests*. 2026. (URL) (visited on 10/30/2026).

[18] Mind.ua. *Banks Must Agree on Open API Standards in Spring 2024 — NBU*. In Ukrainian. 2023. (URL) (visited on 11/03/2026).

[19] Daniel Fett, Dave Tonge, and Joseph Heenan. *FAPI 2.0 Security Profile (Final)*. OpenID Foundation. 2025. (URL) (visited on 11/01/2026).

[20] Brian Campbell et al. *RFC 8705 — OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens*. 2020. (URL) (visited on 11/01/2026).

[21] Torsten Lodderstedt et al. *RFC 9126 — OAuth 2.0 Pushed Authorization Requests*. 2021. (URL) (visited on 11/01/2026).

[22] Ronald L. Rivest and Adi Shamir. «PayWord and MicroMint: Two Simple Micropayment Schemes». In: *Security Protocols: International Workshop, Cambridge, 1996*. Vol. 1189. Lecture Notes in Computer Science. Springer, 1997, pp. 69–87. DOI: 10.1007/3-540-62494-5_6. (URL) (visited on 11/03/2026).